



第二章 密码基础

目 录

「01」

知识点五：安全性概念

「02」

知识点六：安全性定义

「03」

知识点七：安全性证明

「04」

知识点八：安全性模型

「05」

知识点九：通用可组合安全*

1. 安全参数

在密码学中，通常会碰到这么一个名词：安全参数（Security Parameter）。这是用来衡量一个攻击方（adversary）攻破一个加解密机制有多困难的方式。直观的理解就是，安全参数越大，对应的破解加密系统的难度也就越大。

安全参数有两种类型：计算安全参数和统计安全参数。

- 计算安全参数：计算安全参数（computational，通常使用符号 λ 表示）表示攻击者在算力有限（bounded computational power）的情况下，应用离线计算破解一个问题的困难程度，例如破解一个加密算法的困难程度。

计算安全参数 λ 定义了计算的数值空间大小，通常是用bit位数表示。更大的 λ 也就对应了所需要计算的次数更大，所花费的时间越长。

1. 安全参数

在密码学中，通常会碰到这么一个名词：安全参数（Security Parameter）。这是用来衡量一个攻击方（adversary）攻破一个加解密机制有多困难的方式。直观的理解就是，安全参数越大，对应的破解加密系统的难度也就越大。

安全参数有两种类型：计算安全参数和统计安全参数。

- 统计安全参数：统计安全参数（statistical，通常使用符号 σ 表示）表示攻击方者在算力无限（unbounded computation power）的情况下，破解加密机制的困难程度。

统计安全参数依赖于不同变量分布之间的统计距离。直观上说，如果两个变量之间的统计距离越小，则越难区分猜测值和目标变量值。如果一个协议是统计安全的，那么其属于信息论安全，意指无论攻击方算力强弱，也无法破解系统。

2. 完美安全

如果一个具有无限计算能力的敌手从给定的密文中不能获取明文的任何有用信息，则说明这个加密体制具有完美安全性或信息论安全性。

在1949年，香农提出了完美密码模式（定义在对称密码机制上），称为香农理论：

香农理论

设 $(\text{KeyGen}, \text{Enc}, \text{Dec})$ 是一种在明文域 M 上的加密模式，且 $|M| = |K| = |C|$ ，则当且仅当：

- I. 所有密钥 $k \in K$ 都以 $\frac{1}{|K|}$ 的概率通过 KeyGen 生成；
- II. 对所有明文 $m \in M, c \in C$ ，都存在单独密钥 $k \in K$ 使得 $\text{Enc}_k(m)$ 输出 c 。

一次一密是达到完美安全的设计方法，它要求每次均使用随机变化的密钥进行加解密。这样，即使攻击者获取信道上的秘密消息，由于其不知道当前使用的密钥，也无法进行解密。

3. 计算安全

虽然一次一密理论上是安全的，但是它带来的密钥管理等开销使其难以在大多数现实场景中广泛应用。香农定理告诉我们，如果一个加密体制具有完美保密性，那么密钥数量至少要和明文数量一样多。因受香农定理制约，一次一密要求密钥长度至少要和明文一样长，用它加密1GB的视频文件时，密钥也至少得是1GB。

□ 事实上，不需要实现一个绝对安全的密码系统，只需要保证密码算法的破解对攻击者来说是“计算上不可行”的即可，也就是计算安全性。

□ 对敌手的计算安全性需要考虑以下两个方面：

- (1) 攻击的运行时间；
- (2) 攻击的成功概率。

3. 计算安全

假设一个密钥有 λ 比特长，那么最暴力的攻击方式是攻击者需要尝试 2^λ 种可能的密钥。这里的 λ 就是安全参数， λ 的增大意味着攻击者暴力破解的难度呈指数增加。而为了评估 λ 的上限，对敌手计算安全性的考量变更为：（1）渐进运行时间；（2）渐进攻击成功概率。

（1）渐进运行时间

那么如何使得算法的破解是计算上不可行呢？

正如1.2.2节所讲，只需保证算法在“多项式时间”内不能被破解即可。

3. 计算安全

(2) 渐进攻击成功率

密码系统对攻击者的安全性另一方面也要考虑渐进攻击成功率。如果随着 λ 的增加，如果攻击成功率趋近于0，那么即使攻击时间在多么有效率的多项式时间内都无效。为了表示这个思想，就需要使用“可忽略函数”。

可忽略函数是一个极小量，定义如下：

可忽略函数 ν : $\mathbb{N} \rightarrow \mathbb{R}$ 是任意一个趋近于0的速度比任何逆多项式都快的函数。

换句话说，对于任意多项式 p ，除了有限多个 n 以外，均有 $\nu(n) < 1/p(n)$ 。

3. 计算安全

可忽略函数有以下两个性质，即若 $f_1(\lambda)$ 和 $f_2(\lambda)$ 为可忽略函数，则：

1. $f_3(\lambda) = f_1(\lambda) + f_2(\lambda)$ 为可忽略函数；
2. 对所有正多项式 p ，函数 $f_4(\lambda) = p(\lambda)f_1(\lambda)$ 为可忽略函数。

可忽略函数的重要意义在于，如果一个事件是可忽略事件，那么由于它几乎不可能发生，那么就可以被实际目标所忽略。

因此如果对密码系统的某次攻击仅具有可忽略的可能性，那么在设计密码时，就无需考虑这种攻击。

4. 敌手能力

敌手的攻击能力是评判模型安全能力的重要角度之一。

基本攻击模式依照攻击者的攻击能力由弱到强，可分为四种：

- **唯密文攻击（Cipher-only attack）**：最基础的攻击方式，敌手目标是通过观察密文，尝试推测出相应的明文信息。
- **已知明文攻击（Known-plaintext attack）**：敌手已知部分在同一密钥下加密的明文-密文对，敌手的目标是通过已知的明文-密文对，推测出其他一些密文对应的明文信息。

4. 敌手能力

敌手的攻击能力是评判模型安全能力的重要角度之一。

基本攻击模式依照攻击者的攻击能力由弱到强，可分为四种：

- ❑ **选择明文攻击**（Chosen-plaintext attack, CPA）：敌手可以选择获取任意其所选取的明文加密后的结果，敌手目标是通过这些明文信息推测出其他密文对应的明文信息。
- ❑ **选择密文攻击**（Chosen-ciphertext attack, CCA）：敌手甚至可以获得任意其所选择的密文解密后的结果，敌手目标是通过这些信息推测出其他一些（其无法随意选取的）密文对应的明文信息。

有关密码学中的安全性，说法错误的是

- A 如果对密码系统的某类攻击是可忽略事件，那么在设计密码时，就无需考虑这种攻击
- B 通过已知的明文-密文对，推测出其他一些密文对应的明文信息的攻击是已知明文攻击
- C 计算安全性是信息论安全性
- D 一次一密是达到完美安全的设计方法，它要求每次均使用随机变化的密钥进行加解密

目 录

「01」

知识点五：安全性概念

「02」

知识点六：安全性定义

「03」

知识点七：安全性证明

「04」

知识点八：安全性模型

「05」

知识点九：通用可组合安全*

1. 完美保密性

历史上关于第一个关于对称加密方案的安全性的严格定义是香农提出的完美保密性（**perfect secrecy**），发生在上世纪四十年代，如下图。

如果 $\forall m_0, m_1 \in M$ ($|m_0| = |m_1|$) 和 $\forall c \in C$,
有 $\Pr[E(k, m_0) = c] = \Pr[E(k, m_1) = c]$,
则该对称加密体制具有完善保密性，其中 $k \in K$ 是随机的。

这个定义告诉我们，如果一个对称加密方案满足完善保密性，那么密文不会泄露明文的“任何信息”。攻击者窃听到任意一个密文后，既然明文空间中任意两个明文加密后得到该密文的概率都是一样的，那么攻击者自然不可能知道密文对应哪个明文。

1. 一次一密

- 香农指出一次一密能满足完善保密性，而且超级简单。
- 一次一密有很多不同的形式，比特串形式的一次一密是指：加密时，明文和密钥逐比特异或即得密文。解密时，密文和密钥逐比特异或即得明文。

1. 一次一密

有关完美保密性，需要注意的是：

(1) 完美保密性定义在信息论安全上，即拥有无限计算资源。

(2) 对称加密方案的完善保密性考虑的是唯密文攻击，只说明它在唯密文攻击下是安全的，在其他类型的攻击下未必安全。

(3) “任何信息”指的是明文内容的信息。明文长度等信息不包含在内。事实上，现在加密原语即使达到了信息论安全，在使用过程存在的信息泄露（大小模式、访问行为模式等）已经被证明用来破译密码。

(4) 一次一密并不实用，最主要的原因就是，为了达到完善保密性，需要的密钥长度不能短于明文长度，当加密长消息时必须使用长密钥。然而，一次一密的优点也是非常明显的，加解密只需要异或操作，软硬件实现都非常简单，运行速度很快。

1. 一次一密

□ 如何让一次一密更加实用呢？

□ 思路很直接：

在保证安全性的前提下，可以使用短密钥加密长消息。

□ 那么问题又来了，香农定理告诉我们，如果用短密钥加密长消息，就没法达到完善保密性，因为密文势必会泄露明文的信息。

明文的信息都泄露了，那又如何保证安全性呢？

1. 一次一密

□ 我们知道完善保密性可以抵抗无限计算资源的攻击者，但在现实世界中，攻击者的计算资源总是有限的。所以，可以针对有限计算资源的攻击者重新给安全性下一个定义。如果一个加密方案在这个定义下是安全的，那么有限计算资源的攻击者就拿它没办法，这样使用时就足够了。我们称这种加密方案为“计算上安全的”。实际应用中也都是使用这种密码方案。因此，我们的解决方法就是：

设计一种加密方案，可以用短密钥加密长消息；它在面对实际的攻击者时（计算资源有限），没必要达到完善保密性，密文泄露一点儿明文的信息没关系，只要这些信息对攻击者的帮助是可忽略的就行。

2. 不可区分性

如何定义攻击者的能力是可忽略的呢？ 不可区分性已经成为一个重要的基础概念，不可区分安全性也已经成为加密算法的重要安全目标。此外，还有一些其他安全目标，比如不可延展性（鉴于本书内容侧重，不做介绍）、存在不可伪造性等。

通常关于加密算法，定义安全目标为不可区分性或者不可延展性；对于数字签名算法，定义安全目标为存在不可伪造性；对于密钥交换协议，定义安全目标为通过攻击获得的会话密钥和随机数的不可区分性。

2. 不可区分性

令 D_1 和 D_2 为两个以安全参数为索引的概率分布，或表述为 D_1 和 D_2 是以安全参数为输入的两个算法。如果对于所有的算法 A ，存在一个可忽略函数 v ，满足：

$$\Pr[A(D_1(n)) = 1] - \Pr[A(D_2(n)) = 1] \leq v(n)$$

则称 D_1 和 D_2 是不可区分的（Indistinguishable）。

换句话说，当以根据 D_1 或 D_2 采样得到的样本作为输入时，任何一个算法 A 的执行差异都不会超过可忽略函数。

- 如果仅考虑非均匀、多项式时间算法 A ，则此定义描述的是计算不可区分性。
- 如果考虑所有算法而不考虑算法的计算复杂性，则此定义描述的是统计不可区分性。

2. 不可区分性

不可区分安全性。将不可区分性的安全目标与敌手能力结合起来评估模型安全性，已经成为目前主要安全性定义方式之一。在后续讨论中，默认是指计算不可区分性。常见的不可区分安全性定义有两个：

- 1. IND-CPA (indistinguishable chosen-plaintext attack):** 一个具有选择明文攻击能力的敌手对模型进行攻击的结果与随机攻击的结果具有“不可区分性”，则称加密算法在“选择明文攻击下”具有不可区分性，记作“IND-CPA”。
- 2. IND-CCA (indistinguishable chosen-ciphertext attack):** 一个具有选择密文攻击能力的敌手对模型进行攻击的结果与随机攻击的结果具有“不可区分性”，则称加密算法在“选择密文攻击下”具有不可区分性，记作“IND-CCA”。

3. 语义安全性

语义安全是继香农提出的完美保密性之后第二个重要的安全性定义，是Goldwasser和Micali在1984年给出的安全性定义：

如果已知某段未知文段的密文不会泄露任何该文段的其余信息，

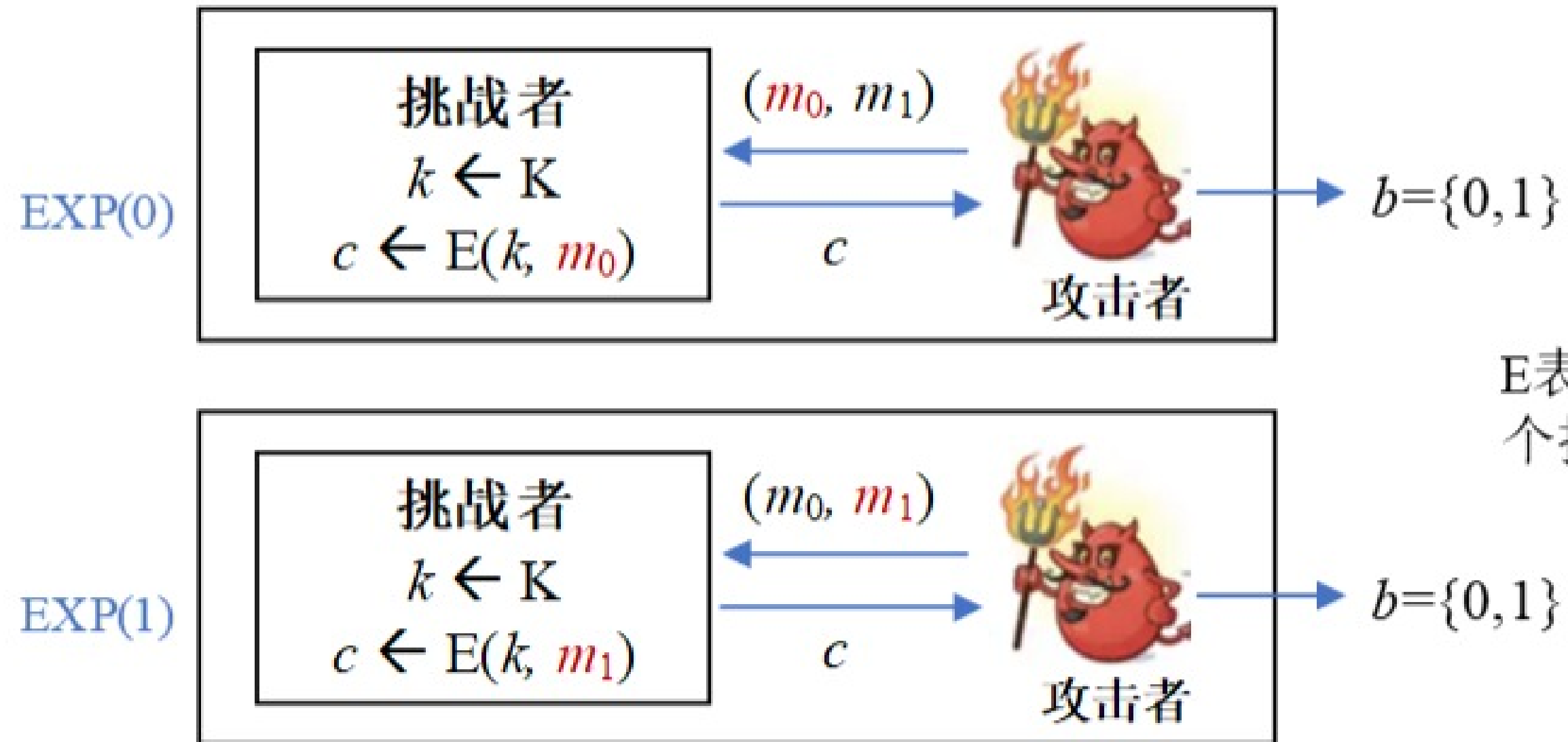
那么则称该密文是语义安全的。

- 语义安全性意味着密文不会向任何计算能力为多项式时间的敌手泄漏有关相应明文的所有有用信息（假定明文长度不被认为是有用信息），它侧重表示被揭露的信息不会被实际窃取。
- Micali等证明语义安全性和选择明文攻击（CPA）安全性是等价的。

3. 语义安全性

- ❑ 在定义语义安全性时，不能像定义完美性加密那样只给出一个概率公式就完事了，必须得把CPA攻击中攻击者“自己选择一些明文”这一活动特点刻画进去。
- ❑ 因此，我们需要借助一个安全模型来帮助给出定义，并通过这些安全模型来证明达到了预期的安全性。

3. 语义安全性



E表示一个对称加密体制，有一个挑战者和一个攻击者A

在上图安全模型中，定义两个黑盒子，每个里面都有一个挑战者，它会随机选择一个密钥，加密一个指定的左侧或右侧的明文，并将密文输出。然后，随机选择一个黑盒子，放在A面前，A并不知道自己面对的是哪一个黑盒子：如果A面对的是上面那个，称之为实验**EXP(0)**；否则，称之为**EXP(1)**。A的任务就是根据得到的密文c猜测自己到底是面对实验EXP(0)还是实验EXP(1)，并输出自己的猜测b。

3. 语义安全性

该安全性模型称为**Left-or-Right (LoR)** 模型，是因为在所设计的实验里，将 m_0 写在左边， m_1 写在右边。详细定义如下：

- ✧ 攻击者A随机选择两个消息 m_0 和 m_1 ， $|m_0|=|m_1|$ ，将 m_0 和 m_1 发送给挑战者。
- ✧ 挑战者从密钥空间 K 中随机选择一个密钥 k ：在实验EXP(0) 中，挑战者将加密左边的明文 m_0 、输出密文 c ；在实验EXP(1) 中，挑战者将加密右边的明文 m_1 、输出密文 c 。
- ✧ 攻击者A根据获得的密文 c 猜测自己面对哪个实验，并输出自己的猜测，记为 b 。比如， $b=0$ 表示攻击者A猜测面对的实验是EXP(0)，加密的消息是 m_0 。

3. 语义安全性

- 注意，在这个对称密码语义安全性的LoR模型中，攻击者A只能给挑战者发送一次明文，也即只允许A询问一次。换句话说，挑战者选择的密钥只使用了一次。
- 很明显，从攻击者的角度，它能区分面对的是哪个实验只能依靠得到的密文 c ，只要加密方案足够好，攻击者就无法从密文 c 获取有用信息来区分两个实验。这就是为什么这两个实验能刻画加密方案安全性的原因。
- 另外， m_0 和 m_1 都是攻击者自己选择的（攻击者愿意选择什么样的明文都可以），这也刻画了选择明文这一特性。

3. 语义安全性

▣ 攻击者优势：为了定义攻击者的优势，我们设 W_0 表示事件攻击者A面对实验EXP(0)时返回1，设 W_1 表示事件攻击者A面对实验EXP(1)时返回1，攻击者A的优势定义为：

$$\text{Adv} = | \Pr[W_0] - \Pr[W_1] |。$$

▣ 基于这个安全模型和攻击者A的优势，正式定义语义安全性如下：

设E是一个对称加密方案，如果所有高效攻击者A在语义安全性的安全模型中的优势都是可忽略的，则E是语义安全的。

直观上可以这么理解，攻击者A总是输出 $b=1$ ，即猜测密文 c 总是来自实验EXP(1)，在这个情况下，EXP(0)和EXP(1)仍然是等价的。或者说，不管这个攻击者面对EXP(0)还是EXP(1)，他输出相同猜测结果（因为 W_0 和 W_1 都只关注攻击者返回1的情况）的概率是相等的。

3. 语义安全性

概率加密

- ▣ 1984年，Micali等提出概率加密的概念，用于设计公钥密码体制。
- ▣ 概率加密指的是，每次加密时都使用独立的随机数用于加密过程，保证了即使用同一个公钥对同一个明文加密所得密文也是以很大概率不相同的。
- ▣ 因此，概率加密是实现IND-CPA安全目标不可缺少的条件。

3. 语义安全性

- Goldwasser和Micali利用概率加密的思想给出了第一个满足IND-CPA安全的公钥加密方案（简称GM方案）。自此，IND-CPA安全的公钥加密方案得到了快速的发展。
- 各种满足不同困难假设的IND-CPA安全的公钥加密方案相继被提出，其中最经典的包括ElGamal方案、Paillier方案以及Damgård-Jurik方案等。
- 回顾RSA方案，很明显RSA没有引入概率加密的概念，在此背景下，相同密钥对同一明文加密的密文是相同的，没有达到IND-CPA安全性。

下列说法错误的是

- A 实际应用中使用的密码体制主要是达到了计算安全性
- B 不可区分安全性是加密算法的重要安全目标
- C 语义安全性和选择明文攻击（CPA）安全性是等价的
- D RSA方案达到了语义安全性

目 录

「01」

知识点五：安全性概念

「02」

知识点六：安全性定义

「03」

知识点七：安全性证明

「04」

知识点八：安全性模型

「05」

知识点九：通用可组合安全*

1. 流密码

流密码基于一次一密基础做了改造，引入伪随机生成器（PRG, pseudo-random generator），它利用短密钥产生长密钥，然后用这个长密钥再和明文异或得到密文。

流密码。定义 $G: K \rightarrow \{0,1\}^n$ 为伪随机函数发生器，加密过程为 $E(k, m) = G(k) \oplus m$ ；解密过程为 $D(k, c) = G(k) \oplus c$ 。很明显，这里使用PRG的输出 $G(k)$ 代替了原来的随机密钥。这种改造后的一次一密被称为流密码（stream cipher）。可以看出， k 在PRG中是种子，而在流密码中作为密钥使用。

流密码本质上是简单套用PRG构造而成的，很明显，流密码的安全性必然依赖于PRG的安全性。流密码的安全性可以这样来理解：只要PRG的输出能以假乱真，也即能够和等长的随机序列不可区分，那么流密码加密出的密文也能以假乱真，也即和一次一密加密出的密文是不可区分的。这就是流密码为什么是安全的依据。

1. 流密码

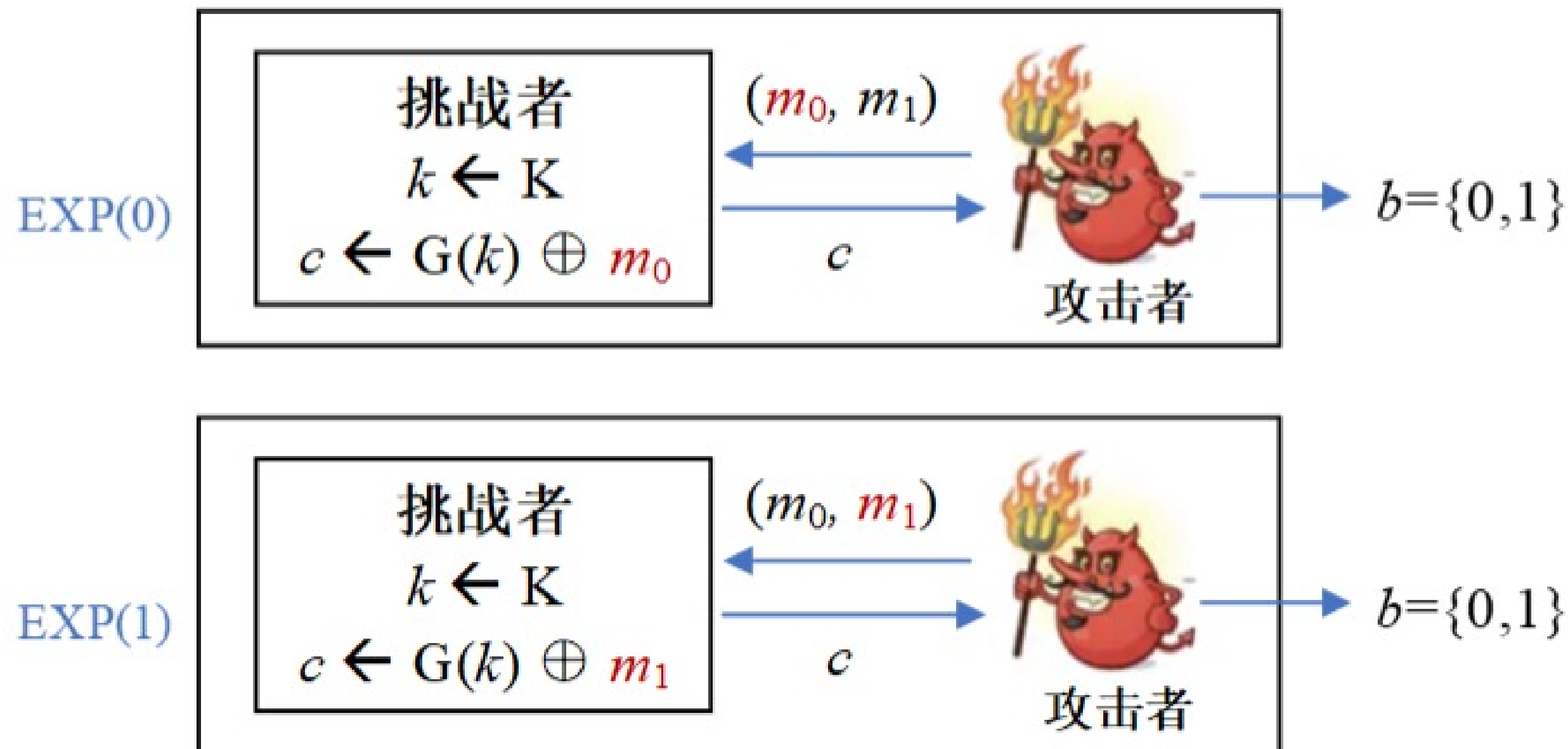
定理：如果 $G: K \rightarrow \{0, 1\}^n$ 是一个安全的PRG，由它构造的流密码便具有语义安全性，其中 $K=\{0,1\}^s$ 。

证明：令E表示一个对称加密体制，G表示一个安全的伪随机生成器，有一个挑战者和一个攻击者A，定义如下两个实验EXP(0)和EXP(1)：

- EXP(0)：攻击者A随机选择两个消息 m_0 和 m_1 ， $|m_0|=|m_1|$ ，将 m_0 和 m_1 发送给挑战者；挑战者从K中随机选择一个种子 k ，计算 $G(k) \oplus m_0$ ，输出左明文的密文 c 。
- EXP(1)：攻击者A随机选择两个消息 m_0 和 m_1 ， $|m_0|=|m_1|$ ，将 m_0 和 m_1 发送给挑战者；挑战者从K中随机选择一个种子 k ，计算 $G(k) \oplus m_1$ ，输出右明文的密文 c 。

设 W_0 表示事件攻击者A面对EXP(0)时输出1，设 W_1 表示事件攻击者A面对EXP(1)时输出1，攻击者A的优势定义为： $Adv=|\Pr[W_0]-\Pr[W_1]|$ 。如果A的优势是可忽略的，则E是语义安全的。

1. 流密码

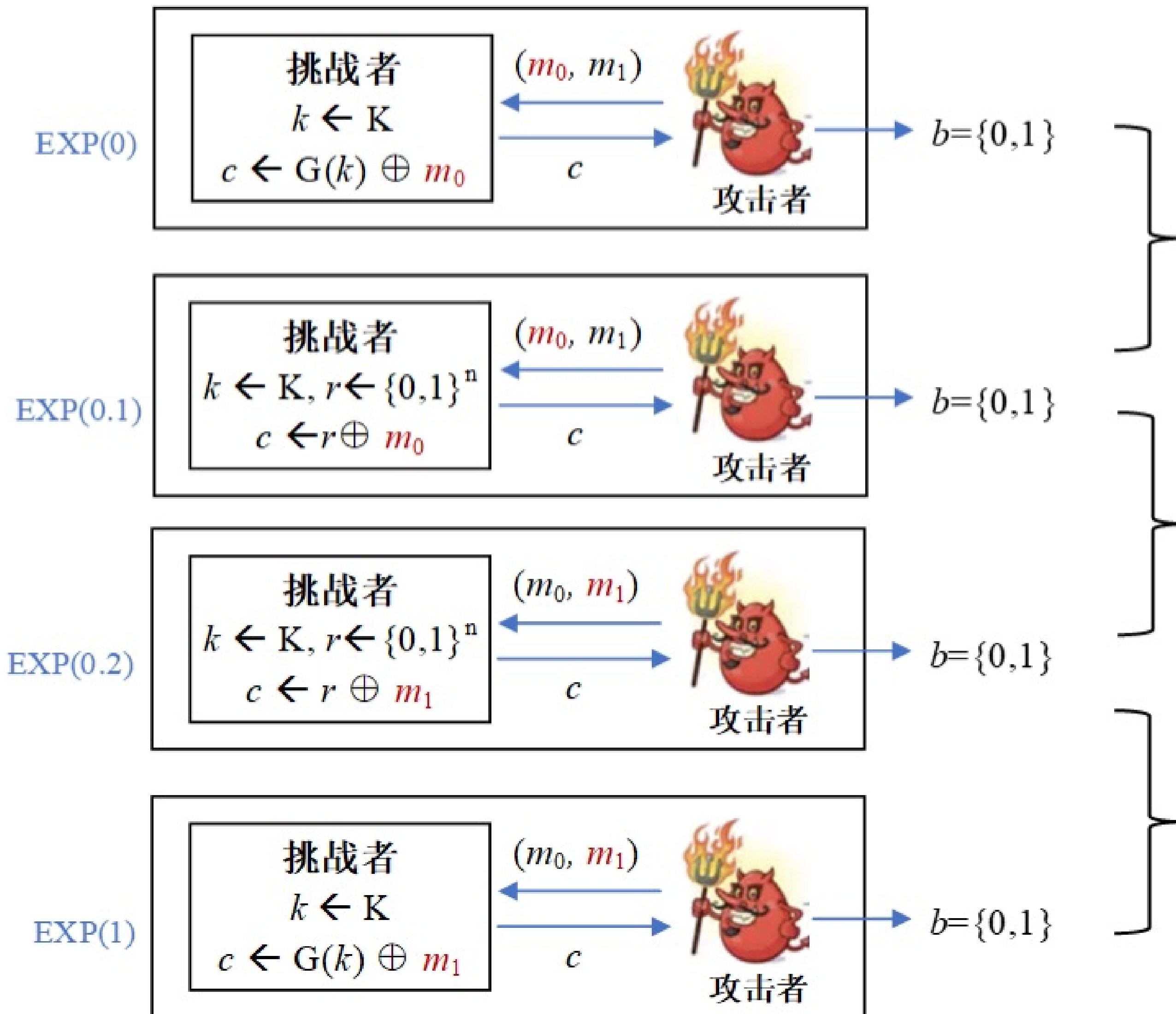


证明思路。 直接证明EXP(0)和EXP(1)是不可区分的不好弄，所以采用间接的证明方法。间接的证明方法是这样的：要证明EXP(0)和EXP(1)是不可区分的，先把EXP(0)这个实验进行一系列改造，逐步改造成EXP(1)。每次改造都会产生一个新的中间实验，我们需要证明改造前的实验和改造后的实验室是不可区分的。最后利用“计算上不可区分”的性质：传递性，推导出EXP(0)和EXP(1)是不可区分的。

1. 流密码

为完成证明，进一步定义如下实验：

- ✧ 实验EXP(0.1)：攻击者A随机选择两个消息 m_0 和 m_1 ， $|m_0|=|m_1|$ ，将 m_0 和 m_1 发送给挑战者；挑战者从 $\{0,1\}^n$ 中随机选择一个随机数 r ，计算 $r \oplus m_0$ ，输出左明文的密文 c 。
- ✧ 实验EXP(0.2)：攻击者A随机选择两个消息 m_0 和 m_1 ， $|m_0|=|m_1|$ ，将 m_0 和 m_1 发送给挑战者；挑战者从 $\{0,1\}^n$ 中随机选择一个随机数 r ，计算 $r \oplus m_1$ ，输出右明文的密文 c 。



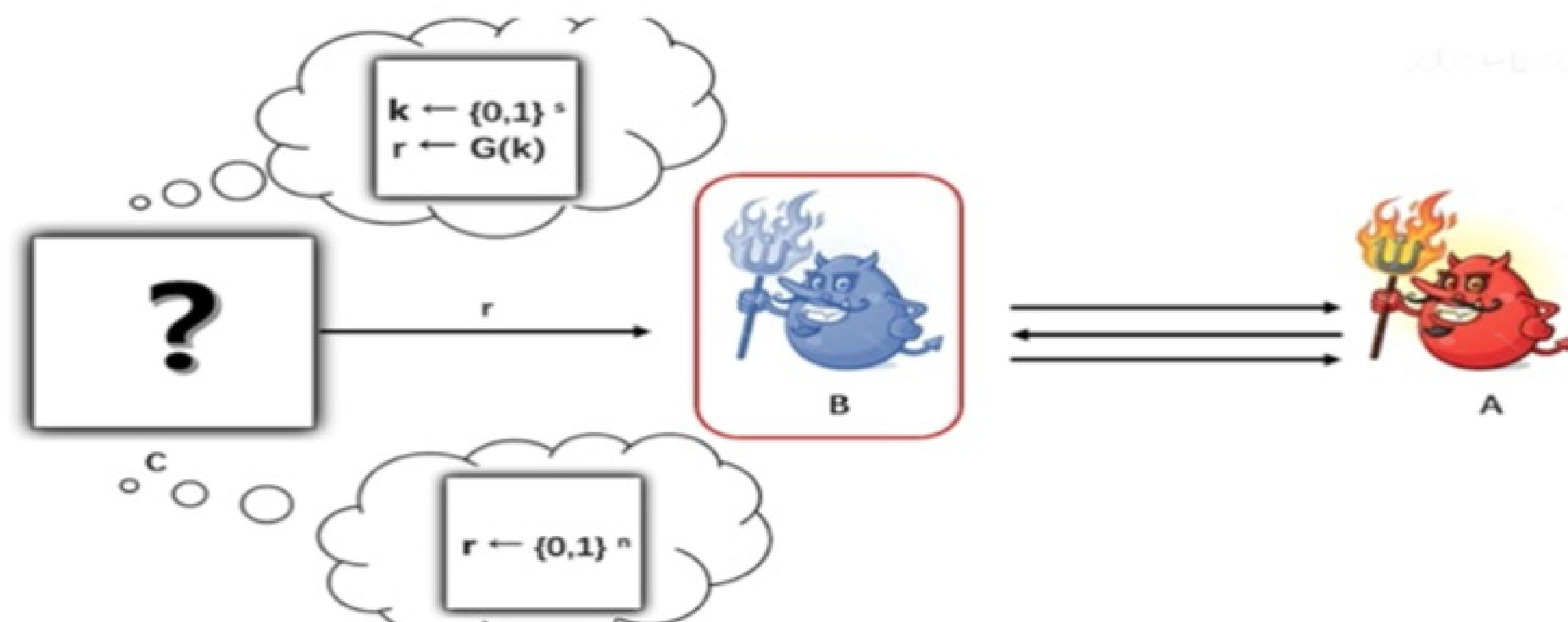
1. 流密码

引理1：如果G是安全的PRG，则EXP(0)和EXP(0.1)是计算上不可区分的。



证明思路。 EXP(0) 和 EXP(0.1) 不可区分的依据是G必须是安全的PRG，也即G(k)与随机序列必须计算上不可区分。不难看出，我们要证明的是：如果G是安全的PRG，则EXP(0)和EXP(0.1)是计算上不可区分的。反证法的实质就是通过**前提假设“EXP(0)和EXP(0.1)是可以区分的”**，以推出矛盾的结论“G不是安全的PRG”。相当于，有一个PRG安全性模型里的挑战者C，它给了一串序列r。r可能等于G(k)，也可能等于随机序列。你需要想办法设计一个高效攻击者，记为**B**，它能以不可忽略的优势将G(k)和随机序列区分开。

1. 流密码



怎么做呢？根据前提假设， $\text{EXP}(0)$ 和 $\text{EXP}(0.1)$ 是可以区分的，那必然存在一个高效攻击者A能以不可忽略的优势区分 $\text{EXP}(0)$ 和 $\text{EXP}(0.1)$ 。你需要做的就是把A作为子程序，让B去调用A，以利用A区分 $\text{EXP}(0)$ 和 $\text{EXP}(0.1)$ 的能力，去识别 r 到底是 $G(k)$ 还是随机序列。如何设计B就是反证法的核心，证明的实质就是写出B的具体执行过程，就像写算法的伪代码一样。证明的最后还需要算出B的优势和时间复杂度：B的优势必须是不可忽略的，同时B必须是高效的。既然存在一个高效攻击者B，它能以不可忽略的优势区分 $G(k)$ 和随机序列，就说明G不是安全的PRG，这正好与题设中“G是安全的PRG”相矛盾，由此证明完毕。

1. 流密码

引理1：如果 G 是安全的PRG，则EXP(0)和EXP(0.1)是计算上不可区分的。

证明：假设存在一个概率多项式时间的攻击者A，它能以不可忽略的优势 Adv_A 区分EXP(0)和EXP(0.1)。挑战者C给出一个长度是 n 的比特序列 r 。我们设计一个概率多项式时间的算法B，它的任务是通过调用A为子程序，在接收到 r 以后，能以不可忽略的优势 Adv_B 识别出 r 是 $G(k)$ 还是随机序列。

在收到挑战者C发来的 r 以后，算法B的执行过程如下：

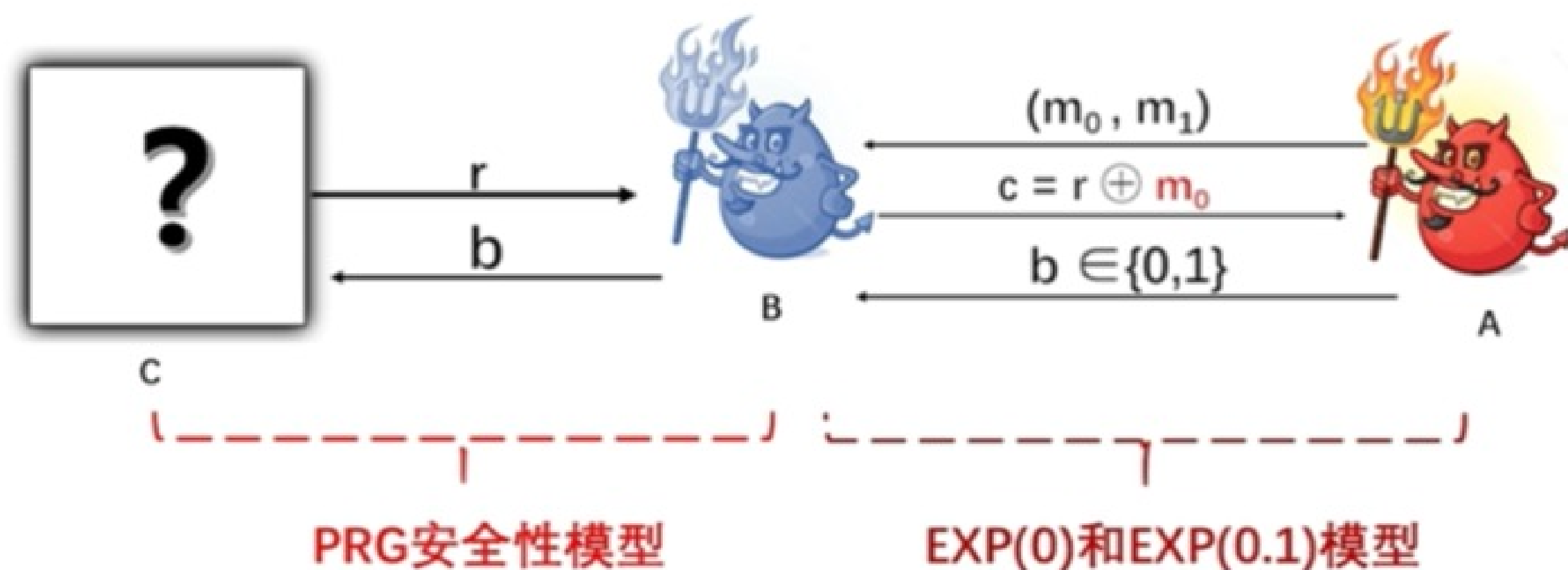
1) 收到A发来的 m_0 和 m_1 后，**计算 $c = r \oplus m_0$** ，并将 c 返回给A作为应答。

很明显，当 $r=G(k)$ 时， $c = G(k) \oplus m_0$ ，A处于EXP(0)；当 r 是随机序列时， $c = \{0,1\}^n \oplus m_0$ ，A处于EXP(0.1)。

2) **A 输出 $b \in \{0,1\}$ 作为自己的猜测**。如果 $b=0$ ，表示A猜测自己处于EXP(0)；否则($b=1$)，表示A猜测自己处于EXP(0.1)。

3) B 将 b 直接返回给C作为自己的猜测。如果 $b=0$ ，表示B猜测 $r=G(k)$ ；否则($b=1$)，表示B猜测 r 是随机序列。

1. 流密码



很明显，因为**B**仅仅是做了一个异或运算，其余都是对**A**的调用，所以 $\text{Adv}_A = \text{Adv}_B$ ，且**B**的时间复杂度等于**A**的时间复杂度。因为假定**A**是一个可以区分两个实验的攻击者，所以**B**同样能在多项式时间内以不可忽略的优势区分 $G(k)$ 和随机序列。这与**G**是安全的**PRG**相矛盾。得证！

1. 流密码

引理2： $\text{EXP}(0.1)$ 和 $\text{EXP}(0.2)$ 是计算上不可区分的。

证明。因为两个实验都是利用随机数 r 异或左明文或者右明文，本质就是一次一密。根据一次一密的完善保密性，任何攻击者区分 $\text{EXP}(0.1)$ 和 $\text{EXP}(0.2)$ 的优势都等于零。

引理3： 如果 G 是安全的PRG，则 $\text{EXP}(0.2)$ 和 $\text{EXP}(1)$ 是计算上不可区分的。

证明。参考引理1，利用反证法证明。

归纳总结。考虑到

(1) $\text{EXP}(0)$ 和 $\text{EXP}(0.1)$ 之间是计算上不可区分的。

(2) $\text{EXP}(0.1)$ 和 $\text{EXP}(0.2)$ 之间是计算上不可区分的。

(3) $\text{EXP}(0.2)$ 和 $\text{EXP}(1)$ 之间是计算上不可区分的。

根据(1)、(2)和(3)以及传递性可知， $\text{EXP}(0)$ 和 $\text{EXP}(1)$ 是计算上不可区分的。

所以，任意高效攻击者区分 $\text{EXP}(0)$ 和 $\text{EXP}(1)$ 的优势都是可忽略的。

小结

- (1) 基于不可区分性的目标，通过设计不可区分性实验，定义安全模型
- (2) 通过反证法、规约法等方式，来证明不可区分性
- (3) 如果难以直接证明，可以拆分为不同实验，逐步传递，来完成证明

目 录

「01」

知识点五：安全性概念

「02」

知识点六：安全性定义

「03」

知识点七：安全性证明

「04」

知识点八：安全性模型

「05」

知识点九：通用可组合安全*

1. 语义安全性模型

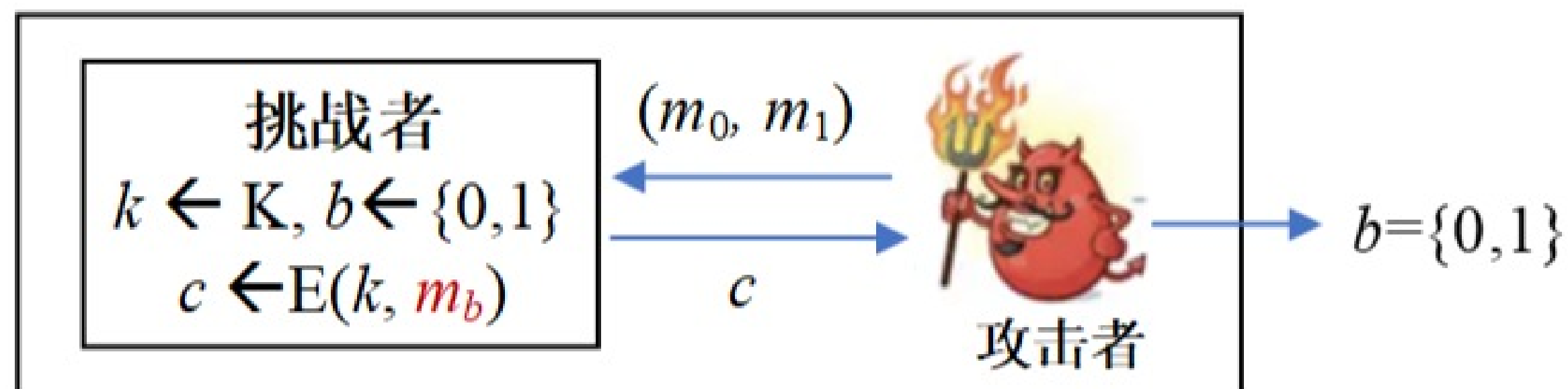
常用的语义安全性模型有三类：

- LOR模型
- 比特随机猜测模型
- Real-Random模型

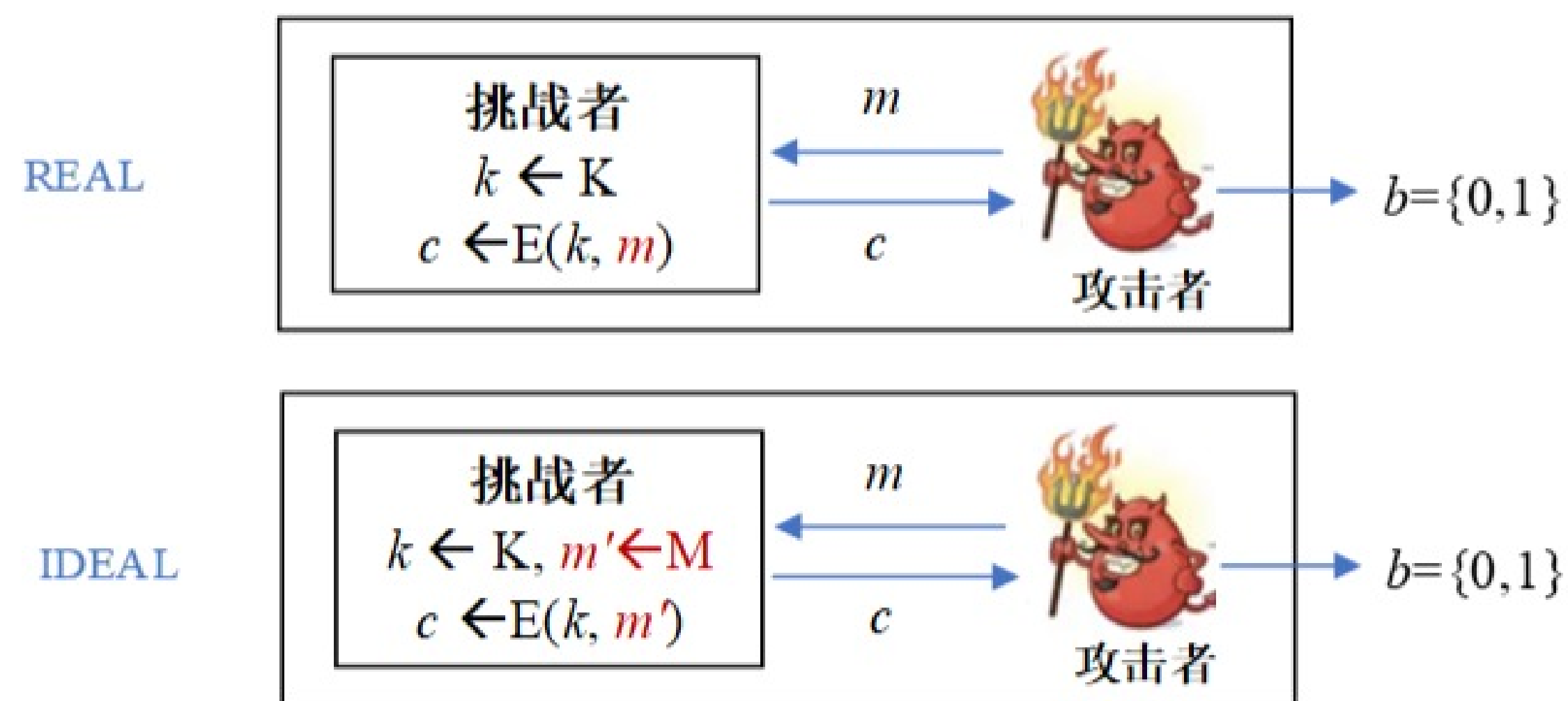
这些模型之间都是等价的。也就是说，一个加密方案在其中一个模型下证明是安全的，它在另一个模型下肯定也能证明是安全的。

- 前面介绍的语义安全的安全性模式是LoR模型，即Left-or-Right模型，即攻击者猜测是对左侧密文还是右侧密文的加密的一种攻击者能力的定义模型。LoR模型还有一种完全对应的比特随机猜测模型。比特随机猜测模型更为常用。

1. 语义安全性模型



比特随机猜测模型。这个模型里只有一个实验。挑战者随机选择一个比特，记为 b 。在收到攻击者发来的两个消息后，挑战者用随机产生的密钥 k 加密消息 m_b ，产生密文 c 。攻击者收到的密文 c ，他需要猜测挑战者选择的 b 到底等于0还是等于1，并输出自己的猜测，记为 b' 。在这个安全性模型中，攻击者的优势定义为 $|\Pr[b=b']-1/2|$ ，很容易理解，攻击者一直猜测相同结果，在随机猜测的情况下概率是1/2。如果不比1/2多，那么意味着攻击者成功的概率是可忽略的。



Real-Random模型。在这个模型里，攻击者只给挑战者发一个消息 m ，挑战者收到 m 以后，会随机决定到底是直接加密 m 还是再产生一个等长的随机 m' 进行加密。这个随机产生的消息记为 m' 。之后，挑战者用随机产生的密钥 k 加密 m 或者 m' ，然后将密文发送给攻击者。

也就是说，攻击者收到的密文可能是他发送的明文 m 的密文，也可能是攻击者随机选择的 m' 的密文。因此，攻击者处于两个不同的实验中，当密文是明文 m 的加密的时候，它处于的是Real实验，当密文是随机消息 m' 的加密的时候，它处于的是Ideal实验。注意，这里称为Ideal实验而不是随机实验。最后，攻击者输出 b ，如果 $b=0$ ，则表示攻击者猜测在于Real实验；否则，处于Ideal实验。

2. 随机预言机模型

- 如前所述，计算安全将攻击者的能力限定为多项式时间，一个方案是否计算安全取决于攻击者成功的优势能否规约到以不可忽略的概率解决某个已知困难问题，或者所有高效攻击者A在相应的安全模型中的优势都是可忽略的。
- 在公钥密码学研究中，早期计算安全的密码方案一般是基于标准模型（Standard Model）下，该模型首先对方案中攻击者的能力加以定义，而且必须强调攻击者一定是自适应性的。然后假设该攻击者成功的概率为某个多项式时间不可忽略的值，然后通过一定的步骤利用该攻击者，将攻击者的能力转化为攻破某已知困难问题的优势。由于该困难问题在多项式时间下无法求解，因而可以得出存在攻击者以不可忽略概率攻破方案这一假设与事实相矛盾。

2. 随机预言机模型

- 标准模型下的密码方案其实就是在**不借助任何假想模型**下所设计的方案，没有任何安全证明环节上的假想存在，仅仅建立在一些被广泛接受的假设基础上，安全性值得高度信赖。不幸的是，基于标准模型的密码方案往往需要大量的计算，难以实用。
- 如何设计一个面向具体应用的密码方案，同时平衡可证明安全性和实用性，成为密码方案设计中首要考虑的问题。因为一个低效率的方案与不安全的方案一样，都无法在实际当中被广大用户接受并广泛使用。

2. 随机预言机模型

随机预言机（Random Oracle, RO）是描述哈希函数安全性的一个启发式模型，由Bellare和Rogaway在1993年首先提出。

- 随机预言机的基本思想是将哈希函数看作公开的理想随机函数。在随机预言机模型中，所有参与方都可以访问用状态预言机实现的公开函数 $H: \{0,1\}^* \rightarrow \{0,1\}^k$ 。给定字符串 $x \in \{0,1\}^k$ ， H 查找自身的调用记录。
- 如果之前从未调用过 $H(x)$ ，则 H 随机选择 $r_x \in \{0,1\}^k$ ，记录输入/输出对 (x, r_x) ，并返回 r_x 。
- 如果之前调用过 $H(x)$ ，则 H 返回 r_x 。预言机通过这一方式实现了一个随机选择函数 $\{0,1\}^* \rightarrow \{0,1\}^k$ 。

2. 随机预言机模型

随机预言机模型是一个启发式模型，因为此模型只能覆盖将哈希函数 H 视为黑盒的攻击算法。随机预言机模型将公开函数（例如像SHA-256这样的标准哈希函数）视为固有随机对象，但现实中不存在这样的公开函数。事实上，可以构造出在随机预言机模型下安全，但当 H 被任意具体函数实例化后便不安全的方案（Canetti et al., 1998）。尽管有这些缺点，但实际应用中通常都可以接受随机预言机模型。如果能假设存在随机预言机，一般都可以设计出更高效的方案。

目 录

「01」

知识点五：安全性概念

「02」

知识点六：安全性定义

「03」

知识点七：安全性证明

「04」

知识点八：安全性模型

「05」

知识点九：通用可组合安全*

1. 安全多方计算

前面介绍的安全性定义和证明过程，主要是面向简单密码协议，交互次数少、形式较为单一。但是，目前很多安全协议设计复杂，比如安全多方计算。

- 安全多方计算问题可以用数学形式化如下： n 个协议参与者 $\{P_1, \dots, P_n\}$ 执行某个协议 π ，每个协议参与者提供秘密输入 x_i ，协议 π 计算函数 $F(x_1, \dots, x_n) = (y_1, \dots, y_n)$ ，结果协议参与者 P_i 应该得到（并且仅仅得到）他的结果 y_i ，除此以外，他不应该知道任何其它敏感信息，比如其他参与者的输入。
- 如果假设有可信第三方 T 存在，这个问题的解决是十分容易的，参与者只需将自己的输入保密传送给 T ，由 T 计算这个函数，然后将计算的结果广播给每一个参与者，这样每个参与者都得到了正确的结果，同时自己的输入也是保密的。然而在现实的应用中，很难找到这样一个所有参与者都信任的 T ，因此安全多方计算的研究主要是针对无可信 T 的情况下，如何安全地计算一个约定函数的问题。从另一个技术角度说，安全多方计算协议实际上就是在努力仿真一个有可信 T 的协议，虽然没有可信 T ，但是希望达到有可信 T 存在的“效果”。

1. 安全多方计算

- ❑ 安全多方计算是许多密码学协议的基础，从广义上讲，所有的密码学协议都是安全多方计算的一个特例，这些密码学协议都可以看作是一组参与者之间存在着各种各样的信任关系(最弱的信任关系就是互不信任)，他们希望通过交互或者非交互的操作来完成一项工作(计算某个约定的函数)。这些协议的不同之处在于协议计算的函数是不一样的。针对类似复杂协议并行运行或者作为其它协议的子协议时整个系统的安全情况，2001年，Canetti等提出了通用可组合安全（UC安全，Universally Composable Security）的概念。
- ❑ UC安全的最优秀的性质就是一种**模块化设计思想**：可以单独设计协议，只要协议满足UC安全，那么就可以保证和其它协议并行运行的安全。
- ❑ UC安全框架的核心由三个模型：现实模型、理想模型以及**F-混合模型**搭建而成，它的主要证明和技术手段是“仿真”。

2. 现实-理想范式

□ 定义安全性时，很自然的想法是列举一个“安全检查清单”，枚举出哪些情况属于违反安全性要求。这种安全性定义方式不仅非常烦琐，而且很容易出现错误。很难说明“安全检查清单”是否枚举出了所有的安全性要求。

□ **现实-理想范式避免采取这种安全性要求描述方式，而是引入了一个定义明确、涵盖所有安全性要求的“理想世界”，通过论述现实世界与理想世界的关系来定义安全性。**

2. 现实-理想范式

- **理想世界。** 在理想世界中，每个参与方 P_i 秘密地将自己拥有的私有输入 x_i 发送给一个**完全可信的参与方 T** ，由后者来安全地计算函数 $F(x_1, \dots, x_n)$ ，并返回结果给所有参与方。通常， **F 被称为理想函数 (Ideal Functionality)**，它扮演了一个不可破可信第三方的角色，能完成协议所执行的功能。
- **完全可信第三方的存在**使得理想世界只是一个想象中的世界。**我们用理想世界作为判断实际协议安全性的基准。**

2. 现实-理想范式

- **现实世界。** 现实世界中不存在可信参与方。在现实世界中，攻击者可以攻陷参与方。在协议开始执行之前就被攻陷的参与方与原始参与方就是攻击者是等价的。根据威胁模型的定义，攻陷参与方可以遵循协议规则执行协议，也可以偏离协议规则执行协议。
- 如果攻击者实施攻击后，其在现实世界中达到的攻击效果与其在理想世界中达到的攻击效果相同，则可以认为现实世界中的协议是安全的。
换句话说，协议的目标是（在给定一系列假设的条件下）使其在现实世界中提供的安全性与其在理想世界中提供的安全性等价。

3. 半诚实安全性

- 半诚实 (Semi-honest) 攻击者可以攻陷参与方，但会遵循协议规则执行协议。换句话说，攻陷参与方会诚实地执行协议，但可能会尝试从其他参与方接收到的消息中尽可能获得更多的信息。
- **半诚实攻击者也被称为被动 (Passive) 攻击者**，因为此类攻击者只能通过观察协议执行过程中自己的视角来尝试得到秘密信息，无法采取其他任何攻击行动。
- **半诚实攻击者通常也被称为诚实但好奇 (Honest-but-curious) 攻击者。**
- 参与方的视角 (View) 包括其私有输入、协议执行期间收到的所有消息所构成的消息列表等。攻击者的视角包含所有攻陷参与方的混合视角。攻击者从协议执行过程中得到的任何信息都必须能表示为以其视角作为输入的高效可计算函数的输出。

3. 半诚实安全性

- 根据现实-理想范式，为了证明协议是安全的，**在理想世界中的攻击者必须能够生成一个视角，此视角与真实世界中的攻击者视角不可区分。**
- 请注意，**理想世界中的攻击者视角只包含发送到 $\mathcal{I}$ 的输入和从 $\mathcal{I}$ 接收到的输出。**
- 因此，理想世界中的攻击者必须能够使用这些信息生成一个视角，此视角和真实世界中的攻击者视角看起来一样。因为此攻击者在理想世界中生成了一个真实世界中的“仿真”攻击者视角，我们称此**攻击者为仿真者 (Simulator)**。
- 能说明存在这样一个仿真者，就能证明攻击者在现实世界中实现的所有攻击效果都可以在理想世界中实现。

3. 半诚实安全性

“现实-理想范式”形式化定义。令 π 为一个协议， F 为一个功能函数。令 C 为攻陷参与方集合，令 Sim 为一个仿真者算法。定义下述两个随机变量的概率分布：

✧ **$\text{Real}_\pi(\kappa, C; x_1, \dots, x_n)$** : 在安全参数 κ 下执行协议，其中每个参与方 P_i 都将使用自己的私有输入 x_i ，诚实地执行协议。令 V_i 为参与方 P_i 的最终视角，令 y_i 为参与方 P_i 的最终输出。

输出 $\{V_i | i \in C\}, (y_1, \dots, y_n)$ 。

✧ **$\text{Ideal}_{F, \text{Sim}}(\kappa, C; x_1, \dots, x_n)$** : 计算 $(y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_n)$ 。

输出 $\text{Sim}(C, \{(x_i, y_i) | i \in C\}), (y_1, \dots, y_n)$ 。

如果现实世界中攻陷参与方所拥有的视角和理想世界中攻击者所拥有的视角不可区分，那么协议在半诚实攻击者的攻击下是安全的。

3. 半诚实安全性

定义 给定协议 π ，如果存在一个仿真者 Sim ，使得对于攻陷参与方集合 C 的所有子集，对于所有的输入 $x_1, \dots, x_n$ ，概率分布

$$\text{Real}_\pi(\kappa, C; x_1, \dots, x_n)$$

和

$$\text{Ideal}_{F, \text{Sim}}(\kappa, C; x_1, \dots, x_n)$$

是（在 κ 下）不可区分的，则称此协议在半诚实攻击者存在的条件下安全地实现了 F 。

初看半诚实攻击模型，简单地读取和分析收到的消息看起来几乎根本就不是一种攻击方法！有理由怀疑是否有必要考虑如此受限的攻击模型。实际上，构造半诚实安全的协议并非易事。更重要的是，在构造更复杂环境下可抵御更强大攻击者攻击的协议时，一般都在半诚实安全协议的基础之上进行改进。此外，很多现实场景确实可以与半诚实攻击模型相对应。

3. 半诚实安全性

仿真证明。接下来将详细描述最关键技术手段，也就是如何利用仿真建立起现实模型和理想模型之间的桥梁，将现实模型的安全规约到理想模型的安全。

- 假设协议A和协议B完成同样的功能，如果攻击者攻击协议A不能比攻击协议B获得更多的信息或者影响，那么协议A至少和协议B一样安全。如果攻击者攻击现实模型下的一个协议 π ，不比攻击理想模型下的一个**Ideal Functionality F**获得更大的影响或者更多的信息，那么 π 至少和F一样安全。
- 以上的说法形式化地表述为：如果任何现实模型攻击者A都存在一个理想模型攻击者S（仿真器），对于任何输入，在现实模型下运行包含A的协议 π 的全局输出，它在理想模型下运行包含S的F的全局输出是不可区分的，那么 π 至少和F一样安全。

3. 半诚实安全性

- 这里的仿真是这样完成的，S在得到F后，它就仿真一个虚拟的现实协议所执行的每个动作，它必须能够完成现实模型攻击者A的所有动作（比如收买参与者），以及看到所有A能看到的信息。
- 因为在理想模型下不存在现实的协议 π ，它只有一个可信第三方F，所以协议中参与者所有的交互和输出都需要虚构（仿真），如果这些虚构（仿真）的信息和现实模型下真实的协议 π 不可区分，那么仿真就成功了，那么也就将现实模型的安全规约到了理想模型的安全。

3. 半诚实安全性

例子。这里用一个例子（Diffie-Hellman协议）解释一下，假设攻击者只是被动的，也就是仅仅窃听协议。

□ Diffie-Hellman协议的现实模型下的执行情况如下

- ✓ 共同输入： p 为一个素数， g 为 F_p^* 的生成元。
- ✓ 输出： Alice和Bob共享 F_p^* 的一个元素。
 - Alice随机选择 $a \in [1, p-1)$ ，发送 $g_a = g^a \bmod p$ 给Bob；
 - Bob随机选择 $b \in [1, p-1)$ ，发送 $g_b = g^b \bmod p$ 给Alice；
 - Alice计算 $k = g_b^a \bmod p$ ；
 - Bob计算 $k = g_a^b \bmod p$ 。

3. 半诚实安全性

□ 定义Diffie-Hellman协议的Ideal Functionality F_{DH} 的描述如下：

Alice和Bob为了得到一个共享密钥，它们向一个 F_{DH} 发送自己的身份标识后， F_{DH} 从 F_p^* 随机选取一个安全的整数 k 发送给Alice和Bob作为Alice和Bob之间的共享密钥，从而完成了现实模型下的Diffie-Hellman协议所需要的功能。

3. 半诚实安全性

仿真器 S_{DH} 的仿真过程如下：

- S_{DH} 仿真**Alice**的动作，随机选择 $a' \in [1, p-1)$ ，发送 $g_{a'} = g^{a'} \bmod p$ 给虚构的**Bob**；
 - S_{DH} 仿真**Bob**的动作，随机选择 $b' \in [1, p-1)$ ，发送 $g_{b'} = g^{b'} \bmod p$ 给虚构的**Alice**；
 - 虚构的**Alice**计算 $k' = g_{b'}^{a'} \bmod p$ （ k' 也就是 F_{DH} 的输出）；
 - 虚构的**Bob**计算 $k' = g_{a'}^{b'} \bmod p$ 。
- 显然这里仿真的虚假消息 $a', b', g_{a'}, g_{b'}, k'$ (全局输出)和真实的 a, b, g_a, g_b, k' 是不可区分的，两者都是落在同一区间的均匀分布。于是我们得出对于被动攻击者而言Diffie-Hellman协议是安全的。
- 本例的仿真是很自然和简单的，但是实际上在很多情况下仿真是很困难的，这是一种构造式的证明，仿真的成败与否直接决定了所设计协议的安全与否。

4. 恶意安全性

恶意（**Malicious**）攻击者，或称主动（**Active**）攻击者，可以让攻陷参与方任意偏离协议规则执行协议，以破坏协议的安全性。恶意攻击者分析协议执行过程的能力与半诚实攻击者相同，但恶意攻击者可以在协议执行期间采取任意行动。请注意，这意味着攻击者可以控制或操作网络，或在网络中注入任意消息。

□ 与之前类似，恶意攻击者场景下的安全性也将通过比较理想世界和现实世界的差异来定义，但需要考虑两个重要的附加因素。

（1）对诚实参与方输出的影响。攻陷参与方偏离协议规则执行协议，可能会对诚实参与方的输出造成影响。例如，攻击者的攻击行为可能会使两个诚实参与方得到不同的输出，但在理想世界中，所有参与方都应该得到相同的输出。在半诚实攻击模型下，这种情况相对来说比较容易解决——虽然安全性定义也要比较现实世界和理想世界的输出，但在半诚实攻击模型下，诚实参与方得到的输出与攻击者（攻陷参与方集合）的攻击行为无关。此外，我们不能也不应该相信恶意攻击者一定会给出最终的输出，因为恶意参与方可以输出任何想输出的结果。

4. 恶意安全性

(2) 输入提取。由于诚实参与方会遵循协议规则执行协议，因此可以明确定义诚实参与方的输入，并在理想世界中将此输入提供给 T 。相反，在现实世界中我们无法明确定义恶意参与方的输入，这意味着在理想世界中我们需要知道将哪个输入提供给 T 。直观上看，对于一个安全的协议，无论攻击者可以在现实世界中实施何种攻击行为，此攻击行为应该也可以通过为攻陷参与方选择适当的输入，从而在理想世界中实现。因此，我们让仿真者选择攻陷参与方的输入。这方面的仿真过程称为输入提取，因为仿真者要从现实世界的攻击者行为中提取出有效的理想世界输入，来“解释”此输入对现实世界造成的影响。大多数安全性证明只需考虑黑盒仿真过程，即仿真者只能访问现实世界中实现攻击的预言机，不能访问攻击代码本身。

4. 恶意安全性

形式化定义。当用 A 表示攻击程序时，我们用 $\text{corrupt}(A)$ 表示被现实世界中的攻击者 A 攻陷的参与方集合，用 $\text{corrupt}(\text{Sim})$ 表示被理想世界中的攻击者 Sim 攻陷的参与方集合。与定义半诚实安全性的方式类似，我们定义现实世界和理想世界的概率分布，并定义一个安全协议，使这两个概率分布满足不可区分性：

- $\text{Real}_{\pi,A}(\kappa; \{x_i | i \notin \text{corrupt}(A)\})$: 在安全参数 κ 下执行协议，其中每个诚实参与方 P_i （即对于所有的 $i \notin \text{corrupt}(A)$ ）使用给定的私有输入 x_i 诚实地执行协议，而攻陷参与方的消息将由 A 选取（将 A 看作被攻陷参与方的“下一条消息”函数）。令 y_i 表示每个诚实参与方 P_i 的输出，令 V_i 表示参与方 P_i 的最终视角。

输出 $(\{V_i | i \in \text{corrupt}(A)\}, \{y_i | i \notin \text{corrupt}(A)\})$ 。

- $\text{Ideal}_{F,\text{Sim}}(\kappa; \{x_i | i \notin \text{corrupt}(A)\})$: 执行 Sim ，直至其输出一个输入集合 $\{x_i | i \in \text{corrupt}(A)\}$ 。计算 $(y_1, \dots, y_n) \leftarrow F(x_1, \dots, x_n)$ 。随后，将 $\{y_i | i \in \text{corrupt}(A)\}$ 发送给 Sim 。令 V^* 表示 Sim 的最终输出（输出是参与方的仿真视角集合）。

输出 $(V^*, \{y_i | i \notin \text{corrupt}(\text{Sim})\})$ 。

4. 恶意安全性

定义 给定协议 π ，如果对于任意一个现实世界中的攻击者 A ，存在一个满足 $\text{corrupt}(A) = \text{corrupt}(\text{Sim})$ 的仿真者 Sim ，使得对于诚实参与方的所有输入 $\{x_i | i \notin \text{corrupt}(A)\}$ ，概率分布

$$\text{Real}_{\pi,A}(\kappa; \{x_i | i \notin \text{corrupt}(A)\})$$

和

$$\text{Ideal}_{F,\text{Sim}}(\kappa; \{x_i | i \notin \text{corrupt}(\text{Sim})\})$$

是（在 κ 下）不可区分的，则称此协议在恶意攻击者存在的条件下安全地实现了 F 。

需要注意的是，该定义仅描述了诚实参与方的输入 $\{x_i | i \notin \text{corrupt}(A)\}$ 。攻陷参与方与现实世界 Real 交互时不需要提供任何输入。而在与理想世界 Sim 交互时，攻陷参与方的输入是间接确定的（仿真者需要根据攻陷参与方的行为来选择将何种输入发送给 F ）。虽然也可以在现实世界定义攻陷参与方的输入，但此输入仅仅是一个“建议”，因为攻陷参与方可以在执行协议时选择使用任何其他的输入（甚至使用与真实输入不一致的输入执行协议）。

4. 恶意安全性

交互功能函数。在理想世界中，功能函数仅包含一轮交互过程：提供输入，给出输出。可以进一步扩展 F 的行为方式，令 F 与参与方进行多轮交互，且在多轮交互的过程中保持其内部状态的私有性。我们称此类功能函数为交互功能函数（Reactive Functionality）。

交互功能函数的一个实例是扑克游戏中的发牌方。此功能函数必须追踪所有扑克牌的状态，获取输入命令，并通过多轮交互向所有参与方提供输出。

另一个交互功能函数实例是承诺（Commitment），这是一个非常常见的功能函数。此功能函数从 P_1 处接收一个比特值 b （更一般的情况是接收一个字符串），告知 P_2 已“承诺” b ，并在内部记住 b 。稍后，如果 P_1 向该功能函数发送命令“披露”（或“打开”），此功能函数将 b 发送给 P_2 。

4. 恶意安全性

可中止安全性。在几乎所有基于消息的2PC协议中，一个参与方会在另一个参与方之前得到最终的输出。如果此参与方是恶意的攻陷参与方，它可以简单地拒绝将最后一条消息发送给诚实参与方，从而阻止诚实参与方得到输出。然而，这种攻击行为与我们之前描述的理想世界攻击行为不兼容。在理想世界中，如果攻陷参与方可以从功能函数中得到输出，则所有参与方均可以得到输出。此性质称为输出公平性（**Output Fairness**）。并非所有的功能函数在计算过程都可以满足输出公平性（Cleve, 1986; Gordon et al., 2008; Asharov et al., 2015a）。

- ❑ 为在恶意攻击场景下覆盖此攻击行为，学者们提出了一种稍弱的安全性定义，称为可中止安全性（**Security with Abort**）。为此，需要按照下述方式稍微修改一下理想功能函数。
- ❑ 首先，允许功能函数得知攻陷参与方的身份。
- ❑ 其次，修改后的功能函数需要一些交互能力：当所有参与方提供输入后，功能函数计算输出结果，但只将输出结果交付给攻陷参与方。
- ❑ 随后，功能函数等待来自攻陷参与方的“交付”或“中止”命令。收到“交付”命令后，功能函数将输出交付给所有诚实参与方。收到“中止”命令后，功能函数向所有诚实参与方交付一个表示协议中止的输出（ $\perp$ ）。

4. 恶意安全性

适应性攻陷。在我们已经定义的现实世界和理想世界中，哪些参与方是攻陷参与方在整个交互过程中是固定不变的。我们称满足这一安全模型的协议在静态性攻陷（**Static Corruption**）下是安全的。

还可以考虑另一个场景，攻击者在协议执行期间可以根据交互过程中得到的信息选择攻陷哪些参与方。我们称这一攻击行为是适应性攻陷（**Adaptive Corruption**）。

□ 可以在现实-理想范式中为适应性攻陷攻击行为建立安全模型，方法是允许攻击者发出形式为“攻陷 P_i ”的命令。在现实世界中，这会令攻击者得到 P_i 的当前视角（包括 P_i 的内部私有随机状态），并接管其在协议执行过程中发送消息的控制权。在理想世界中，仿真者只能得到攻陷此参与方时该参与方的输入和输出，必须使用这些信息生成仿真视角。显然，各个参与方的视角是相互关联的（如果 P_i 向 P_j 发送一条消息，则此消息会同时包含在两个参与方的视角中）。适应性安全的挑战是仿真者必须逐段生成攻陷参与方的视角。例如，当参与方 P_i 被攻陷时，我们要求仿真者生成 P_i 的视角。仿真者必须在未知 P_j 私有输入的条件下仿真出 P_j 发送给 P_i 的所有消息。随后，仿真者可能需要提供 P_j 的视角来“解释”之前发送的协议消息与 P_j 的私有输入是匹配的。

5. 组合性

出于模块化考虑，设计协议时经常会让协议调用其他的理想功能函数。例如，我们可能需要设计一个安全实现某功能函数 F 的协议 π 。在 π 中，参与方除了彼此要发送消息之外，还需要与另一个功能函数 G 交互。因此，该协议在现实世界中包含 G ，但在理想世界（一般来说）仅包含 F 。我们称这一修改后的现实世界为 G -混合世界。

- ▣ 对安全模型的一个很自然的要求是组合性（**Composition**）：如果 π 是一个安全实现 F 的 G -混合协议（即 π 的参与方需要彼此发送消息，且需要与一个理想的 G 交互），且 ρ 是一个安全实现 G 的协议，则以最直接的方式组合使用 π 和 ρ （将调用 G 替换为调用 ρ ）应该可以得到安全实现 F 的协议。但是，一些非常直观的组合使用方式并不能保证多个安全协议可以安全组合、满足可组合性！
- ▣ 保证组合性的标准方式是使用Canetti提出的通用可组合性（Universal Composability, UC）框架（Canetti, 2001）。UC框架在我们之前描述的安全模型上进行了扩展，在安全模型中增加了一个称为环境（**Environment**）的实体，此实体也同时包含在理想世界和现实世界中。引入环境实体的目的是体现协议执行时的“上下文”（例如，当前协议被某个更大的协议所调用）。环境实体为诚实参与方选择输入，接收诚实参与方的输出。环境实体可以与攻击者进行任意交互。

谢 谢

